

```
CREATE TABLE USERS (  
    ID SERIAL PRIMARY KEY,  
    NAME VARCHAR(100),  
    EMAIL VARCHAR(150),  
    IS_ACTIVE BOOLEAN  
);
```

```
CREATE TABLE DEPARTMENTS (  
    ID SERIAL PRIMARY KEY,  
    NAME VARCHAR(100) UNIQUE NOT NULL  
);
```

```
CREATE TABLE USERS (  
    ID SERIAL PRIMARY KEY,  
    NAME VARCHAR(100) NOT NULL,  
    EMAIL VARCHAR(150) UNIQUE NOT NULL,  
    ROLE VARCHAR(50) NOT NULL CHECK (ROLE IN ('student', 'professor', 'admin')),  
    DEPARTMENT_ID INT REFERENCES DEPARTMENTS (ID) ON DELETE SET NULL,  
    METADATA JSONB,  
    IS_ACTIVE BOOLEAN DEFAULT TRUE,  
    CREATED_AT TIMESTAMP DEFAULT NOW()  
);
```

```
SELECT  
    *  
FROM  
    PG_TABLES  
WHERE  
    SCHEMANAME = 'public';
```

```
DROP TABLE USERS;
```

--Create the Courses (Products) Table: Create a courses table with an id, title, and fee (using a numeric type suitable for currency).

-- Sweet Hint: For the fee, NUMERIC(10, 2) is a great data type choice to handle decimal values safely.

-- single record insert

```
INSERT INTO  
    DEPARTMENTS (NAME)  
VALUES  
    ('Computer Science');
```

```
ON CONFLICT (NAME) DO NOTHING;
```

```
SELECT
    *
FROM
    USERS;
```

```
-- multiple Record Insert
INSERT INTO
    DEPARTMENTS (ID, NAME)
VALUES
    (2, 'Mechanical Engineering'),
    (3, 'Electronics & Communication'),
    (4, 'Business Administration'),
    (5, 'Basic Sciences'),
    (6, 'Computer Science')
ON CONFLICT (NAME) DO NOTHING;
```

```
CREATE TABLE COURSES (
    ID SERIAL PRIMARY KEY,
    TITLE VARCHAR(200) NOT NULL,
    FEE NUMERIC(8, 2) NOT NULL CHECK (FEE >= 0),
    INSTRUCTOR_ID INT REFERENCES USERS (ID) ON DELETE SET NULL,
    CREATED_AT TIMESTAMP DEFAULT NOW() -- ← add this
);
```

```
-- Run this command to wipe the tables and reset the ID counters:
-- (Note: CASCADE safely handles the foreign key links during the wipe, and RESTART
IDENTITY ensures the next insert starts back at id = 1!)
TRUNCATE TABLE USERS,
COURSES RESTART IDENTITY CASCADE;
```

```
-- 1: The Basic SELECT *
-- Let's see all the 100 students, professors, and admins we imported yesterday.
```

```
SELECT
    *
FROM
    USERS;
```

```
SELECT
    NAME AS "Full Name",
FROM
    USERS
WHERE
    ROLE = 'professor';
```

-- 2: Selecting Specific Fields (The Right Way)

```
SELECT
    NAME,
    EMAIL,
    ROLE
FROM
    USERS;
```

-- 3: Formatting Output with AS (Aliases)

-- Let's make that output look professional for a report. Notice the double quotes because our aliases have spaces:

```
SELECT
    NAME AS "Full Name",
    EMAIL AS "Contact Address"
FROM
    USERS;
```

-- 4: Finding Unique Values with DISTINCT

-- Imagine you forget what roles exist in your system. If you run `SELECT role FROM users;`, you get 100 rows repeating "student" and "professor".

```
SELECT DISTINCT
    ROLE
FROM
    USERS;
```

-- 5: The "Wow" Factor - Math on the Fly

-- Let's say college management decides to add a 13% GST (tax) to all course fees. You don't need to update the database; you can calculate it directly in the SELECT!

```
SELECT
    *
FROM
    COURSES;
```

```
SELECT
    TITLE,
    FEE AS "Base Fees",
    FEE * 1.13 AS "Total with Ta"
FROM
    COURSES;
```

-- task

```
SELECT
    'Profile: ' || ROLE || ' - ' || NAME AS "User Badge"
FROM
```

USERS;

-- UPDATE STATEMENT

-- Scenario 1: The Basic Update (Fixing a Status)

-- Let's say a student named Priya Singh (email = 'priya.s@student.com') dropped out, so her is_active status was false. She just re-enrolled! Let's update her status:

SELECT

*

FROM

USERS

WHERE

EMAIL = 'priya.s@student.com';

UPDATE USERS

SET

IS_ACTIVE = TRUE

WHERE

EMAIL = 'priya.s@student.com';

-- Scenario 2: Multi-Field Update with RETURNING

-- Prof. Sunita Sharma just got promoted to an Admin and got a new official email. We need to update two fields at once and return the row to confirm it worked.

SELECT

*

FROM

USERS

WHERE

NAME = 'Prof. Sunita Sharma';

UPDATE USERS

SET

EMAIL = 'sunita.sharma@admin.com',

ROLE = 'admin'

WHERE

NAME = 'Prof. Sunita Sharma'

RETURNING

ID,

NAME,

EMAIL,

ROLE;

-- Scenario 3: The "Dynamic Math" Update

-- The college board has decided to increase the fee by 10% for any course that currently costs less than 3000. We don't need to manually calculate anything; PostgreSQL does the math for us!

```
SELECT
    *
FROM
    COURSES
WHERE
    FEE <= 3000;

UPDATE COURSES
SET
    FEE = FEE * 1.10
WHERE
    FEE <= 3000
RETURNING
    ID,
    TITLE,
    FEE;
```

-- Scenario 4: The Catastrophic Mistake (Demonstrate this live!)
-- Show your audience what happens if they forget the WHERE clause.

```
UPDATE USERS
SET
    IS_ACTIVE = TRUE
WHERE
    EMAIL = 'priya.s@student.com';
```

-- DELETE
-- 1: A student named Akash Basnet dropped out. We don't want to lose his past fee payment records, so we hide him instead of deleting him.

```
SELECT
    *
FROM
    USERS
WHERE
    NAME = 'Akash Basnet';

UPDATE USERS
SET
    IS_ACTIVE = FALSE
WHERE
    EMAIL = 'akash.b@student.com';
```

-- 2: Single Row "Hard Delete" with RETURNING
-- A spam user somehow registered as a student. We need them completely removed from the system forever, and we want to see exactly who we just deleted to confirm.

DELETE FROM USERS

-- 3: Batch Deletion (Cleaning up the Database)
-- The academic year is over. The college administration wants to permanently remove every single student or professor whose account is currently marked as inactive.

-- 4: The Danger Zone (Forgetting WHERE)
-- Show your audience exactly what happens if they write:

-- 5: TRUNCATE vs. DELETE

-- DELETE FROM payments; is slow because PostgreSQL logs every single row it deletes.
TRUNCATE is the professional way to wipe a table clean instantly:

/*

Precision Filter using WHERE

*/

-- Scenario 1: The Exact Match (=)
-- We need to find the specific profile for a student to load their dashboard.

SELECT

*

FROM

USERS;

SELECT

ID,
NAME,
EMAIL

FROM

USERS

WHERE

EMAIL = 'rahul.s@student.com';

-- Scenario 2: Numeric Logic (> and <=)
-- The college wants a list of all premium courses that cost 4000 or more.

SELECT

*

FROM

COURSES

WHERE

FEE >= 4000;

-- Scenario 3: The Strict AND Operator

-- We need to find all active users who are professors. Both conditions must be met!

```
SELECT
    NAME,
    EMAIL,
    IS_ACTIVE,
    ROLE
FROM
    USERS
WHERE
    IS_ACTIVE = TRUE
    AND ROLE = 'professor';
```

```
SELECT
    *
FROM
    USERS;
```

-- Scenario 4: The Flexible OR Operator

-- We are sending an urgent administrative email. We need to grab everyone who is an Admin or a Professor.

```
SELECT
    *
FROM
    USERS
WHERE
    ROLE = 'admin'
    OR ROLE = 'professor';
```

-- Scenario 5: The Exclusion (!=)

-- We want a list of all students, but we want to completely exclude Department 1 (Computer Science) because they have a separate system.

```
SELECT
    NAME,
    DEPARTMENT_ID
FROM
    USERS
WHERE
    ROLE = 'student'
    AND DEPARTMENT_ID != 1;
```

-- Important to take care The Parentheses Trap

-- The college administration wants a list of all inactive students from either Computer Science (Department 1) OR Mechanical Engineering (Department 2). Write the query to fetch their names and emails.

```
SELECT
    NAME,
    DEPARTMENT_ID,
    IS_ACTIVE
FROM
    USERS
WHERE
    (
        DEPARTMENT_ID = 1
        OR DEPARTMENT_ID = 2
    )
    AND IS_ACTIVE = FALSE;
```

```
SELECT
    NAME,
    EMAIL,
    DEPARTMENT_ID
FROM
    USERS
WHERE
    DEPARTMENT_ID IN (2, 4, 5)
WHERE
    FEE >= 2500
    AND FEE <= 4000;
```

```
WHERE
    FEE BETWEEN 2500.00 AND 4000.00;
```

```
SELECT
    TITLE,
    FEE
FROM
    COURSES
SELECT
    NAME,
    CREATED_AT
FROM
    USERS
WHERE
    CREATED_AT BETWEEN '2026-01-01' AND '2026-03-31';
```

```
SELECT
    NAME,
    ROLE
```

```
FROM
    USERS
WHERE
    NAME LIKE '%Sharma%';
```

```
SELECT
    ID,
    NAME,
    EMAIL
FROM
    USERS
WHERE
    NAME ILIKE '%thapa%';
```

-- ADVANCED FILTER

-- Scenario 1: The IN Operator (Replacing messy ORs)

-- The administration needs an email list for all students in the Mechanical (2), Business (4), and Basic Sciences (5) departments.

```
select name, email , department_id from users
where department_id = 2 or department_id = 4 or department_id = 5 ;
```

```
select name, email , department_id from users
where department_id IN (2,4,5);
```

-- Scenario 2: The BETWEEN Operator (Numbers)

-- Let's find all the mid-tier courses priced right in the sweet spot between 2500 and 4000.

```
select title, fee from courses
where fee >= 2500 AND fee <= 4000;
```

```
where fee BETWEEN 2500 AND 4000;
```

```
select * from courses;
```

-- Scenario 3: The BETWEEN Operator (Dates & Timestamps)
-- Let's find all users who registered in the first quarter of the year. Since our created_at column is a TIMESTAMP, we can search date ranges effortlessly.

```
SELECT name, created_at FROM users ;  
WHERE created_at BETWEEN '2026-01-01' AND '2026-03-31';
```

-- Scenario 4: The LIKE Operator (Case-Sensitive Exact Match)
-- We know a professor's last name is "Sharma", and we want to ensure we match that exact capitalization.

```
select name, role from users  
where name LIKE '%Sharma%';
```

-- (The % before and after means: "I don't care what comes before Sharma, and I don't care what comes after. Just find the word Sharma inside the string.")

-- Scenario 5: The ILIKE Operator (Building a UI Search Bar)
-- A user types "thapa" into your Next.js search bar. If you use LIKE, it will fail because your database has "Thapa" with a capital T. You must use ILIKE!

```
SELECT id, name, email FROM users  
WHERE name ILIKE '%thApa%';
```

-- SORTING & PAGINATION

-- 1: Basic Alphabetical Sorting (ASC)
-- The college admin wants a directory of all users, sorted alphabetically from A to Z.
select id, name, email from users
order by name DESC;
order by name ASC;

-- 2: The "Leaderboard" (DESC + LIMIT)
-- We are building a pricing page and want to highlight our 3 most expensive premium courses at the very top.
select fee, title from courses

```
order by fee DESC
LIMIT 3;
```

-- 3: Multi-Column Sorting

-- We want to group users by their roles (Admins, then Professors, then Students), but within those groups, we want their names sorted alphabetically.

```
select role, name from users
order by role ASC, name ASC;
```

-- 4: Building Pagination (The Admin Dashboard)

-- Imagine our Next.js admin dashboard shows a table of users. We only want to show 10 users per page.

-- Page 1: We want the first 10 users, ordered by their ID. We skip zero rows.

```
select id, name, email from users
order by id ASC
limit 10;
```

-- Page 2: The user clicks "Next Page." We want the next 10 users. So we must tell the database to skip the first 10!

```
select id, name, email from users
order by id ASC
limit 10 offset 10;
```

-- Page 3: We skip the first 20.

```
select id, name, email from users
order by id ASC
limit 10 offset 20;
```

--

-- 1: Adding a NOT NULL Rule

-- Let's say we realize that the `is_active` column in our users table currently allows NULL. We want to force it to always be explicitly true or false.

```
alter table users
alter column is_active
set NOT NULL;
```

```
select * from users;
```

```
SELECT constraint_name, column_name
FROM information_schema.constraint_column_usage
WHERE table_name = 'users';
```

-- 2: Creating and Enforcing UNIQUE

-- The college decides to start tracking phone numbers, and no two students can register with the exact same phone number.

-- First, we add the column:

```
alter table users
add column phone_number varchar(20);
```

-- Next, we make it unique. We will name this rule unique_phone_rule:

```
alter table users
add constraint unique_phone_rule UNIQUE (phone_number);
```

-- 3: The Custom CHECK Rule

-- Let's pretend we forgot to secure our payments table during setup. We want to guarantee that nobody can ever insert a payment amount of \$0 or negative numbers.

```
select * from payments;
```

```
CREATE TABLE payments (
  id SERIAL PRIMARY KEY,
  student_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  amount NUMERIC(8,2) NOT NULL CHECK (amount > 0),
  status VARCHAR(50) NOT NULL CHECK (status IN ('pending', 'completed', 'failed')),
  paid_at TIMESTAMP DEFAULT NOW()
);
```

```
INSERT INTO payments (student_id, amount, status)
VALUES
```

```
(9, 4999.00, 'completed'),
(10, 2999.50, 'completed'),
(11, 5999.00, 'pending'),
(12, 3500.00, 'completed'),
(13, 4999.00, 'failed'),
(14, 6500.00, 'completed'),
(15, 3200.00, 'completed'),
(16, 2800.00, 'pending'),
```

```
(17, 4100.00, 'completed'),  
(18, 3800.00, 'failed');
```

```
alter table payments  
add constraint  
positive_payment_check  
CHECK (amount > 0)
```

-- 4: Dropping (Removing) a Constraint
-- Sometimes business rules change. If the college decides they no longer care if phone numbers are unique, you can delete the rule using the custom name you gave it:

```
ALTER TABLE users  
DROP CONSTRAINT unique_phone_rule;
```

-- 1: The Internal Table (SERIAL)
-- For internal college data that is never exposed in a public URL, SERIAL is perfectly fine. Let's create an exams table.

```
CREATE TABLE exams (  
  id SERIAL PRIMARY KEY,  
  title VARCHAR(100) NOT NULL,  
  total_marks INT NOT NULL  
);
```

```
SELECT * FROM exams;
```

-- 2: The Public-Facing Table (UUID)
-- Imagine we are generating digital course certificates for our students. We will send them a public link to share on LinkedIn. We cannot use SERIAL (Certificate ID 5) because people could just guess Certificate ID 6. We must use a UUID!

```
CREATE TABLE certificates (  
  id UUID PRIMARY KEY DEFAULT UUIDV7(),  
  student_id INT REFERENCES users(id),  
  course_id INT REFERENCES courses(id),  
  issued_on TIMESTAMP DEFAULT NOW()  
);
```

-- 3: Testing the UUID Generation

-- Let's insert a certificate to see the UUID magic in action. Notice we don't even mention the id column!

```
INSERT INTO certificates (student_id, course_id)
VALUES (9, 3)
RETURNING *;
```

-- Day 13: Foreign Keys & Relational Thinking

-- 1: Building the Bridge (The enrollments Table)

-- A student can take many courses, and a course has many students. We need a table just to link them together using two Foreign Keys!

```
create table enrollments (
    id bigserial primary key,
    student_id int REFERENCES users(id) on delete cascade,
    course_id INT REFERENCES courses(id) ON DELETE CASCADE,
    enrolled_on TIMESTAMP DEFAULT NOW()
);
```

```
select * from enrollments;
```

-- 2: The Successful Insert

-- Let's enroll Student ID 9 (Aayush) into Course ID 3 (PostgreSQL Bootcamp).

```
insert into enrollments (student_id, course_id) values(9, 3) returning *;
```

-- 3: The "Gotcha" Error (Referential Integrity in Action)

-- Demonstrate exactly why Foreign Keys are amazing. Try to enroll a ghost student (ID 9999) into a course.

```
insert into enrollments (student_id, course_id) values(9, 9999) returning *;
```

-- 4: Adding an FK to an Existing Table

-- Imagine we created a library_books table but forgot to link it to the users table to track who borrowed the book.

```
CREATE TABLE library_books (
```

```
id SERIAL PRIMARY KEY,  
title VARCHAR(200) NOT NULL,  
borrowed_by_user_id INT, -- The column exists, but there is no rule yet!  
borrowed_on TIMESTAMP DEFAULT NOW()  
);
```

```
INSERT INTO library_books (title, borrowed_by_user_id)  
VALUES ('Clean Code by Robert C. Martin', NULL);
```

```
ALTER TABLE library_books  
ADD CONSTRAINT fk_borrower  
FOREIGN KEY (borrowed_by_user_id) REFERENCES users(id);
```

```
select * from library_books;
```

```
UPDATE library_books  
SET borrowed_by_user_id = 9999  
WHERE id = 1;
```

```
SELECT users.name, departments.name AS department_name  
FROM users  
INNER JOIN departments ON users.department_id = departments.id  
WHERE users.role = 'professor';
```

```
INSERT INTO enrollments (student_id, course_id)  
VALUES  
  (10, 1), -- Rahul taking Advanced Next.js  
  (10, 3), -- Rahul also taking PostgreSQL  
  (11, 4), -- Priya taking MERN Stack  
  (12, 1), -- Amit taking Advanced Next.js  
  (12, 2), -- Amit taking Data Structures  
  (13, 5), -- Sneha taking Machine Learning  
  (14, 6), -- Vikram taking Engineering Thermodynamics  
  (15, 7), -- Pooja taking Fluid Mechanics  
  (16, 6), -- Akash taking Engineering Thermodynamics  
  (17, 8), -- Neha taking Automobile Engineering  
  (18, 9), -- Rohan taking Digital Logic  
  (19, 10), -- Ananya taking Microprocessor Systems  
  (20, 3), -- Birat taking PostgreSQL  
  (21, 4), -- Dipendra taking MERN Stack  
  (22, 5); -- Niranjan taking Machine Learning
```

-- 1: The LEFT JOIN (Seeing Everyone)

-- Let's pull a list of all students and their course enrollments. But we want to see every student, even if they haven't picked a class yet.

```
select u.name as student , e.course_id
from users u
left join enrollments e
on u.id = e.student_id
where u.role = 'student';
```

-- 2: The "Ghost Hunter" Query (IS NULL)

-- Now, let's filter that exact list so our Next.js backend can send an automated "Please Enroll" email strictly to students who have zero enrollments.

```
select u.name , u.email
from users u
left join enrollments e
on u.id = e.student_id
where u.role = 'student' AND e.course_id IS NULL;
```

-- 3: The RIGHT JOIN (Flipping the Perspective)

-- We want to see a list of all courses, and see the students enrolled in them. We want to see all courses, even if no students have signed up yet (so we know which courses to market better).

```
SELECT e.student_id, c.title AS course_name
FROM enrollments e
RIGHT JOIN courses c
ON e.course_id = c.id;
```

```
SELECT e.student_id, c.title AS course_name
FROM courses c
LEFT JOIN enrollments e
ON e.course_id = c.id;
```

-- 1: The FULL OUTER JOIN (The Complete Audit)

-- Let's say the college administration wants a complete reconciliation report of Users and Departments. We want to see:

-- Users who belong to a department.

-- Users who do not have a department yet (like new students).

-- Departments that do not have any users assigned to them yet.

```
select u.name as username , d.name as department_name
from users u
full outer join departments d
on u.department_id = d.id
```

```
select * from departments;
```

-- 2: The CROSS JOIN (Merchandise Combinations)

-- The college is launching a merch store. We have a table of 3 shirt colors (Red, Blue, Black) and a table of 4 sizes (S, M, L, XL). We need to generate a list of every possible SKU combination to feed into our frontend dropdown menu.

```
select c.color, s.size
from merch_colors c
cross join merch_size s;
```

--3: The Danger of CROSS JOIN (The Crash)

-- Demonstrate why developers fear this command if used accidentally.

```
select d.name, c.title
from departments d
cross join courses c;
```

-- 1: The COUNT (Headcount)

-- The administration needs to know exactly how many active students are currently enrolled in the college.

```
select count(is_active)
from users
where
```

```
select * from users limit 5;
```

-- 2: The SUM (Total Revenue)

-- Let's check our payments table to see exactly how much money the college has successfully collected.

-- 3: The AVG (Pricing Strategy)

-- We want to know the average price of our premium courses. Let's throw in a cool PostgreSQL trick to round the decimal so it looks like real currency!

-- 4: The MIN and MAX (Price Ranges)

-- We are building a pricing page and want to show a badge that says: "Courses ranging from \$X to \$Y!". We can grab both in one query.

-- 1: Solving the Cliffhanger (GROUP BY)

-- Let's group our users by their role to see exactly how many admins, professors, and students we have in the college.

```
select role, count(id) as "Total Count"
from users
where is_active = true
group by role;
```

-- 2: Revenue per Status (Grouping Money)

-- Let's look at the payments table we built on Day 10. How much money is successfully 'completed' versus still 'pending' or 'failed'?

```
select * from payments;
```

```
select status, sum(amount) as "Total"
```

-- 3: The HAVING Clause (Filtering the Groups)

-- Let's see which departments have a high volume of users. We want to group users by department_id, count them, but only show departments that have more than 5 users.

Updated - 19th June 2026

-- Interview questions

- 1: The High-Value Students (The Original Boss Query)
- The Business Question: The college wants to send a VIP thank-you gift to our top-spending active students.
- The Task: Write a query that finds the total revenue per student.

- Filter 1: Only calculate this for students who are currently `is_active = true`.
- Filter 2: Only show students whose total spent amount is strictly greater than 5000.00.

- Bonus: Sort the list so the highest spender is at the very top.
- Sweet Hint for the Video: You need to INNER JOIN the users and payments tables. Remember the golden rule of filtering: use WHERE for the active status (before grouping) and HAVING for the total spent (after grouping).

```
SELECT u.name AS student_name, SUM(p.amount) AS total_spent
FROM users u
INNER JOIN payments p
ON u.id = p.student_id
WHERE u.is_active = true
GROUP BY u.name
HAVING SUM(p.amount) > 5000.00
ORDER BY total_spent DESC;
```

- 2: The Popular Premium Courses
- The Business Question: We want to know which of our expensive courses are actually popular so we can give bonuses to the professors.
- The Task: Write a query that shows the Course Title, the Professor's Name, and the Total Number of Enrolled Students.
- Filter 1: Only include "Premium" courses that have a fee greater than 3000.00.
- Filter 2: Only show courses that have at least 2 students enrolled (we don't want to see empty or low-performing courses).
- Sweet Hint for the Video: This is a 3-table jump! You need to connect courses, users (for the instructor name), and enrollments (to count the students). Group by both the course title and the professor's name!

```
select * from courses
select * from enrollments;
```

-- 3: The Department Budget Audit

-- The Business Question: The financial department is auditing the average cost of education across different fields. They want to find affordable departments taught by active staff.

-- The Task: Write a query that calculates the mathematical average course fee for each Department.

-- Filter 1: Only include courses taught by professors who are is_active = true (ignore retired professors' courses).

-- Filter 2: Only show departments where the average course fee is strictly less than 4000.00.

-- Bonus: Round the average to 2 decimal places so it looks like real money, and sort it from cheapest to most expensive.

-- Sweet Hint for the Video: Another 3-table join! Link departments to users (professors), and then to courses. Wrap your AVG(fee) in a ROUND(..., 2) function for that clean finish.

Updated - 9th July 2026

-- 1: The Basic Two-Table Join (Users & Departments)

-- We want a list of all professors, but instead of seeing department_id = 1, we want to see the actual text "Computer Science." Let's join users with departments.

```
select users.name, departments.name as department_name
from users
INNER JOIN departments
ON users.department_id = departments.id
where role = 'professor'
```

-- 2: Using Aliases (Users & Courses)

-- Typing users and courses over and over gets tedious. Let's use Table Aliases to find out which professor is teaching which course.

```
select users.name, courses.title , courses.fee
from users
```

```
INNER JOIN courses
ON users.id = courses.instructor_id
```

```
select * from courses;
```

```
select u.name, c.title , c.fee
from users u
INNER JOIN courses c
ON u.id = c.instructor_id
```

-- 3: The Three-Table Jump (The Ultimate Query)

-- Let's see what courses our students are actually enrolled in. To do this, we have to start at users, jump through the enrollments mapping table, and land on courses!

```
SELECT u.name AS student, c.title AS course, e.enrolled_on
FROM users u
```

```
INNER JOIN enrollments e
ON u.id = e.student_id
```

```
INNER JOIN courses c
ON e.course_id = c.id;
```

```
select * from enrollments;
```

```
INSERT INTO enrollments (student_id, course_id)
VALUES
  (10, 1), -- Rahul taking Advanced Next.js
  (10, 3), -- Rahul also taking PostgreSQL
  (11, 4), -- Priya taking MERN Stack
  (12, 1), -- Amit taking Advanced Next.js
  (12, 2), -- Amit taking Data Structures
  (13, 5), -- Sneha taking Machine Learning
  (14, 6), -- Vikram taking Engineering Thermodynamics
  (15, 7), -- Pooja taking Fluid Mechanics
  (16, 6), -- Akash taking Engineering Thermodynamics
  (17, 8), -- Neha taking Automobile Engineering
  (18, 9), -- Rohan taking Digital Logic
  (19, 10), -- Ananya taking Microprocessor Systems
  (20, 3), -- Birat taking PostgreSQL
  (21, 4), -- Dipendra taking MERN Stack
```

(22, 5); -- Niranjan taking Machine Learning

-- 1: The WHERE Clause (The "Above Average" Problem)

-- Since our users don't have an age column, let's use the courses table to find premium content. We want to find all courses that cost strictly more than the average course fee.

-- The Inner Query (SELECT AVG(fee) FROM courses) runs first!

```
SELECT title, fee
FROM courses
WHERE fee > (SELECT AVG(fee) FROM courses);
```

```
SELECT AVG(fee) FROM courses
```

-- 2: The IN Clause (Targeted Filtering)

-- Let's find the names and emails of all users who belong to any "Engineering" department (whether it's Mechanical or Civil). Instead of a complex JOIN, we can use a subquery!

```
SELECT name, email
FROM users
WHERE department_id IN (
    SELECT id FROM departments WHERE name ILIKE '%Engineering%'
);
```

-- 1: Refactoring the Subquery (Readability Win)

-- On Day 18, we found courses that cost more than the average fee using a subquery in the WHERE clause. Let's rewrite it using a CTE. It makes the code instantly self-documenting!

```
WITH AverageCourseFee As (
    SELECT AVG(fee) as avg_fee FROM courses
)
```

```
SELECT title, fee
FROM courses
WHERE fee > ( select avg_fee from AverageCourseFee );
```

-- 2: The "Virtual Table" (Aggregating before Joining)
-- We want a list of students and the total amount of money they have spent. But we want to calculate the math before we join the tables to keep things fast and clean.

```
WITH StudentSpending AS (  
    SELECT p.student_id, SUM(p.amount) AS total_spent  
    FROM payments p  
    WHERE p.status = 'completed'  
    group by p.student_id  
)
```

```
SELECT u.id, u.name, s.total_spent  
FROM users u  
INNER JOIN StudentSpending s ON u.id = s.student_id;
```

-- 3: The Multi-Step Pipeline (Multiple CTEs)
-- Let's chain CTEs together! We only want to look at active students, and we only want to look at premium courses. Then, we join them together.

```
WITH ActiveStudents AS (  
    SELECT id, name FROM users WHERE role = 'student' AND is_active = true  
)  
PremiumCourses AS (  
    SELECT id, title FROM courses WHERE fee >= 4000.00  
)
```

```
SELECT a.name AS student, p.title AS course  
FROM enrollments e  
INNER JOIN ActiveStudents a ON e.student_id = a.id  
INNER JOIN PremiumCourses p ON e.course_id = p.id;
```

